



รายงาน PS02: Process Creation & Pipes

เสนอ

อาจารย์ ราชวิชช์ สโรชวิกสิต

จัดทำโดย

นายภูมิพัฒน์ อภิวัตนะพงศ์	67070501035
นายวิรัช ทองอุทัยศรี	67070501041
นายเจษฎา เกียรติกมลวงค์	67070501080

รายงานนี้เป็นส่วนหนึ่งของวิชา CPE 333 Operating Systems
ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
ภาคการศึกษาที่ 1 ปีการศึกษา 2569
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

Contents

สภาพแวดล้อมที่ใช้ในการทดลอง	2
สรุปความคืบหน้าของรายงานฉบับนี้	3
1 fork() แบบมีและไม่มี wait()	4
1.1 Source code	4
1.1.1 1.1 โปรแกรม helloworld (ไม่มี wait())	4
1.1.2 1.2 โปรแกรมที่เพิ่ม wait()	5
1.2 ผลการทดลอง	6
1.3 อภิปรายผล	6
2 Process Status และ sleep()	7
2.1 Source code	7
2.1.1 2.1 Child ตายก่อน โดย Parent ยังทำงานอยู่	7
2.1.2 2.2 Parent ตายก่อน โดย Child ยังทำงานอยู่	8
2.2 ผลการทดลอง	9
2.2.1 2.1 Child ตายแล้ว แต่ Parent ยังทำงานอยู่	9
2.2.2 2.2 Parent ตายแล้ว แต่ Child ยังทำงานอยู่	10
2.3 อภิปรายผล	11
2.3.1 เปรียบเทียบสองสถานการณ์	11
2.3.2 ปรากฏการณ์ <defunct>	11
3 จำนวน process สูงสุดที่ fork ได้	13
3.1 Source code	13
3.2 ผลการทดลอง	13
3.2.1 3.1 การวัดครั้งแรก	13
3.2.2 3.2 เมื่อมีโปรแกรมอื่นทำงานค้างอยู่	13
3.3 อภิปรายผล	13
4 การสื่อสารผ่าน Pipe	15
4.1 Source code	15
4.2 ผลการทดลอง	15
4.3 อภิปรายผล	15
5 การทดลองกรณีพิเศษของ Pipe	16
5.1 5.1 ผู้ส่งพยายามอ่านข้อความของตัวเองกลับ	16
5.2 5.2 ผู้รับอ่านก่อนที่ผู้ส่งจะส่ง	16
5.3 5.3 ผู้ส่งส่งหลายข้อความก่อนที่ผู้รับจะอ่าน	16
5.4 สรุปผลการทดลองข้อ 5	17
สรุปผลการทดลอง	18
เอกสารอ้างอิง	18

สภาพแวดล้อมที่ใช้ในการทดลอง

ระบบปฏิบัติการ	Ubuntu 24.04.1 LTS (บน WSL2 / Windows 11)
Kernel	6.6.87.2-microsoft-standard-WSL2 (ตรวจด้วย <code>uname -r</code>)
คอมไพเลอร์	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
คำสั่งคอมไพล์	gcc -Wall -Wextra -o q1_1 q1_1_helloworld.c gcc -Wall -Wextra -o q1_2 q1_2_wait.c gcc -Wall -Wextra -o q2_1 q2_1_child_first.c gcc -Wall -Wextra -o q2_2 q2_2_parent_first.c
ผลการคอมไพล์	ผ่านทุกไฟล์ ไม่มี warning

หมายเหตุเรื่อง WSL2 WSL2 รัน Linux kernel จริง ดังนั้น `fork()`, `wait()`, `pipe()` และคำสั่ง `ps` ทำงานได้ตามปกติ อย่างไรก็ตามตัวเลขขีดจำกัดจำนวน process ในข้อ 3 อาจต่างจาก Linux ที่ติดตั้งแบบ native หรือแบบ VM เพราะ WSL2 จัดสรรหน่วยความจำให้ VM แบบไดนามิก

สรุปความคืบหน้าของรายงานฉบับนี้

ข้อ	หัวข้อ	โค้ด	ผลการทดลอง
1	fork() แบบมีและไม่มี wait()	เสร็จ	เสร็จ
2	Process status และ sleep() (zombie / orphan)	เสร็จ	เสร็จ
3	จำนวน process สูงสุดที่ fork() ได้	ยังไม่ได้ทำ	ยังไม่ได้ทำ
4	การสื่อสารผ่าน pipe()	ยังไม่ได้ทำ	ยังไม่ได้ทำ
5	กรณีพิเศษของ pipe (5.1–5.3)	ยังไม่ได้ทำ	ยังไม่ได้ทำ

หมายเหตุ ส่วนที่ยังไม่ได้ทำถูกทำเครื่องหมายไว้ด้วยข้อความสีแดง [ยังไม่ได้ทำ: ...] ในเนื้อหารายงาน เพื่อให้หาจุดที่ต้องเติมได้ง่าย ก่อนส่งจริงต้องเติมให้ครบทุกจุดและลบคำสั่ง \todo ออก

1 fork() แบบมีและไม่มี wait()

1.1 Source code

1.1.1 1.1 โปรแกรม helloworld (ไม่มี wait())

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main(void)
6 {
7     printf("Hello World!\n");
8
9     pid_t pid = fork();
10    if (pid < 0) {
11        perror("fork");
12        exit(EXIT_FAILURE);
13    }
14
15    if (pid == 0) {
16        /* child */
17        printf("I am the child, my PID = %d, my parent PID = %d\n",
18              getpid(), getppid());
19    } else {
20        /* parent */
21        printf("I am the parent, my PID = %d, my child PID = %d\n",
22              getpid(), pid);
23    }
24
25    printf("Common line printed by PID %d\n", getpid());
26    return 0;
27 }
```

Code 1: q1_1_helloworld.c – fork() without wait()

คำอธิบายการทำงาน: เรียก fork() เพื่อสร้าง child process หากเป็น child (pid == 0) จะพิมพ์ PID ของตนเองและ parent ส่วน parent จะพิมพ์ PID ของตนเองและลูก เนื่องจากไม่มี wait() parent จึงจบการทำงานโดยไม่รอ child

1.1.2 1.2 โปรแกรมที่เพิ่ม wait()

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main(void)
7 {
8     printf("Hello World!\n");
9
10    pid_t pid = fork();
11    if (pid < 0) {
12        perror("fork");
13        exit(EXIT_FAILURE);
14    }
15
16    if (pid == 0) {
17        /* child */
18        printf("I am the child, my PID = %d, my parent PID = %d\n",
19              getpid(), getppid());
20    } else {
21        /* parent */
22        wait(NULL);
23        printf("I am the parent, my PID = %d, my child PID = %d\n",
24              getpid(), pid);
25    }
26
27    printf("Common line printed by PID %d\n", getpid());
28    return 0;
29 }
```

Code 2: q1_2_wait.c – fork() with wait()

คำอธิบายการทำงาน: เพิ่มการเรียก wait(NULL) ในฝั่ง parent เพื่อสั่งให้ parent หยุดรอ (Blocked) จนกว่า child จะทำงานเสร็จและคืนสถานะเรียบร้อย บังคับให้ลำดับการแสดงผลของ child เกิดขึ้นก่อน parent เสมอ

1.2 ผลการทดลอง

```
$ gcc -Wall -Wextra -o q1_1 q1_1_helloworld.c
$ ./q1_1
Hello World!
I am the parent, my PID = 3642, my child PID = 3643
Common line printed by PID 3642
I am the child, my PID = 3643, my parent PID = 3642
Common line printed by PID 3643

$ ./q1_1
Hello World!
I am the parent, my PID = 3644, my child PID = 3645
Common line printed by PID 3644
I am the child, my PID = 3645, my parent PID = 3632    <-- PPID changed (Orphan)!
Common line printed by PID 3645
```

Code 3: ผลลัพธ์ของข้อ 1.1 (ไม่มี wait())

```
$ gcc -Wall -Wextra -o q1_2 q1_2_wait.c
$ ./q1_2
Hello World!
I am the child, my PID = 3649, my parent PID = 3648
Common line printed by PID 3649
I am the parent, my PID = 3648, my child PID = 3649
Common line printed by PID 3648
```

Code 4: ผลลัพธ์ของข้อ 1.2 (มี wait())

1.3 อภิปรายผล

จากการทดลองเปรียบเทียบระหว่างโปรแกรมข้อ 1.1 (ไม่มี wait()) และข้อ 1.2 (มี wait()) สรุปความแตกต่างที่สำคัญได้ดังนี้

- **การชิงโครโนซ์และลำดับ Output:** ในข้อ 1.1 ทั้ง Parent และ Child แย่งกันทำงานอย่างอิสระ ลำดับการพิมพ์ข้อความจึงขึ้นอยู่กับ Kernel Scheduler และไม่แน่นอน (Non-deterministic) ในขณะที่ข้อ 1.2 การเรียก wait(NULL) จะทำให้ Parent หยุดรอ (Blocked) จนกว่า Child จะทำงานเสร็จ ลำดับ Output จึงมีความแน่นอน (Deterministic) โดยข้อความของ Child จะปรากฏก่อนเสมอ
- **ปรากฏการณ์ Orphan Process:** ในข้อ 1.1 มีโอกาสที่ Parent จะทำงานเสร็จและจบโปรแกรมไปก่อน ทำให้ Child กลายเป็น Orphan Process และถูกเคอร์เนลยกให้ init (หรือ Subreaper) เป็น Parent ใหม่แทน (สังเกตได้จาก PPID ของ Child ที่เปลี่ยนไปในการรันครั้งที่ 2) ส่วนในข้อ 1.2 จะไม่เกิดเหตุการณ์นี้เนื่องจาก Parent รอ Child อยู่เสมอ

2 Process Status และ sleep()

การทดลองนี้ใช้ `sleep()` เป็นเครื่องมือควบคุม *ลำดับการตาย* ของ parent และ child เพื่อสร้างสถานการณ์สองแบบที่ตรงข้ามกัน แล้วใช้คำสั่ง `ps` ตรวจสอบสถานะของ process ในช่วงเวลาที่สนใจ ทั้งสองโปรแกรม **จงใจไม่เรียก** `wait()` เพื่อให้เห็นพฤติกรรมที่เกิดขึ้นเมื่อ parent ไม่เก็บกวาดลูกของตนเอง

2.1 Source code

2.1.1 2.1 Child ตายก่อน โดย Parent ยังทำงานอยู่

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  #define PARENT_LIFETIME 30  /* seconds the parent stays alive after child dies */
6
7  int main(void)
8  {
9      pid_t pid = fork();
10
11     if (pid < 0) {
12         perror("fork");
13         exit(EXIT_FAILURE);
14     }
15
16     if (pid == 0) {
17         /* CHILD: dies almost immediately */
18         printf("[child ] PID = %d, PPID = %d -- exiting right now\n",
19             getpid(), getppid());
20         fflush(stdout);
21         exit(0);
22     }
23
24     /* PARENT: outlives the child but never reaps it */
25     printf("[parent] PID = %d, child PID = %d\n", getpid(), pid);
26     printf("[parent] NOT calling wait(). Sleeping %d s.\n", PARENT_LIFETIME);
27     printf("[parent] Inspect now with:  ps -ef | grep -e q2_1 -e defunct\n");
28     fflush(stdout);
29
30     sleep(PARENT_LIFETIME);
31
32     printf("[parent] PID = %d exiting; the zombie child %d is now re-parented "
33         "to init/systemd and reaped.\n", getpid(), pid);
34     return 0;
35 }

```

Code 5: q2_1_child_first.c – child dies first, parent never reaps

คำอธิบายการทำงาน: หลัง `fork()` ฝั่ง child จะพิมพ์ PID ของตนเองแล้วเรียก `exit(0)` ทันที ส่วน parent เข้าสู่ `sleep(30)` ทำให้เกิดช่วงเวลา 30 วินาทีที่ child ตายไปแล้วแต่ parent ยังมีชีวิตอยู่และยังไม่ได้เรียก `wait()` จึงเป็นช่วงที่เปิดอีก terminal หนึ่งเข้าไปตรวจด้วย `ps` ได้ทัน การเรียก `fflush(stdout)` ก่อน `exit()` และก่อน `sleep()` เป็นการบังคับให้ข้อความถูกเขียนออกทันที ไม่ค้างอยู่ใน buffer

2.1.2 2.2 Parent ตายก่อน โดย Child ยังทำงานอยู่

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  #define CHILD_LIFETIME 30
6
7  int main(void)
8  {
9      pid_t pid = fork();
10
11     if (pid < 0) {
12         perror("fork");
13         exit(EXIT_FAILURE);
14     }
15
16     if (pid == 0) {
17         /* CHILD: outlives the parent */
18         printf("[child ] PID = %d, PPID = %d (original parent, still alive)\n",
19             getpid(), getppid());
20         fflush(stdout);
21
22         sleep(2); /* give the parent time to die */
23
24         printf("[child ] PID = %d, PPID = %d (parent is gone -> re-parented)\n",
25             getpid(), getppid());
26         printf("[child ] Inspect now with: ps -ef | grep q2_2\n");
27         fflush(stdout);
28
29         sleep(CHILD_LIFETIME);
30
31         printf("[child ] PID = %d exiting, final PPID = %d\n",
32             getpid(), getppid());
33         exit(0);
34     }
35
36     /* PARENT: exits immediately, without waiting */
37     printf("[parent] PID = %d, child PID = %d -- exiting immediately\n",
38         getpid(), pid);
39     fflush(stdout);
40     return 0;
41 }

```

Code 6: q2_2_parent_first.c – parent dies first, child becomes an orphan

คำอธิบายการทำงาน: สลับบทบาทจากข้อ 2.1 คือ parent พิมพ์ข้อความแล้วจบการทำงานทันที ส่วน child อยู่ต่ออีก 30 วินาที จุดสำคัญคือ child เรียก `getppid()` สองครั้ง โดยมี `sleep(2)` คั่นกลาง ครั้งแรกพิมพ์ตอน parent ยังมีชีวิต ครั้งที่สองพิมพ์หลัง parent ตายไปแล้ว ทำให้เห็นค่า PPID ที่เปลี่ยนไปได้จากตัวโปรแกรมเองโดยไม่ต้องพึ่ง `ps`

2.2 ผลการทดลอง

2.2.1 2.1 Child ตายแล้ว แต่ Parent ยังทำงานอยู่

```
$ gcc -Wall -Wextra -o q2_1 q2_1_child_first.c
$ ./q2_1 &
[parent] PID = 6908, child PID = 6910
[parent] NOT calling wait(). Sleeping 30 s.
[child ] PID = 6910, PPID = 6908 -- exiting right now

$ ps -ef | grep -e q2_1 -e defunct | grep -v grep
UID      PID    PPID  C STIME TTY      TIME    CMD
poomipat 6908    6900  0 18:58 pts/2    00:00:00 ./q2_1
poomipat 6910    6908  0 18:58 pts/2    00:00:00 [q2_1] <defunct>

$ ps -eo pid,ppid,stat,comm | grep q2_1 | grep -v grep
    PID    PPID STAT  COMMAND
    6908    6900 S+   q2_1      <- parent sleeping
    6910    6908 Z+   q2_1      <- child is a ZOMBIE

$ ps -el | grep q2_1
F S    UID      PID    PPID  C PRI  NI ADDR SZ WCHAN  TTY      TIME CMD
0 S   1000    6908    6900  0  80   0 -   670 hrtime pts/2    00:00:00 q2_1
1 Z   1000    6910    6908  0  80   0 -    0 -      pts/2    00:00:00 q2_1
      ^^ SZ = 0 : all memory already released
```

Code 7: ผลลัพธ์และสถานะจาก ps ในกรณี 2.1

Table 1: สรุปค่าที่สังเกตได้ในกรณี 2.1

รายการ	ค่าที่สังเกตได้
PID ของ parent	6908
PID ของ child	6910
PPID ของ child	6908 (ยังเป็น parent ตัวจริง ไม่ถูก reparent)
สถานะของ child ใน ps	Z+ แสดงผลเป็น [q2_1] <defunct>
หน่วยความจำของ child (SZ)	0

เมื่อรอจนครบ 30 วินาทีและ parent จบการทำงาน ตรวจสอบด้วย ps อีกครั้งพบว่า ไม่เหลือ process ชื่อ q2_1 อยู่ในระบบเลย แสดงว่า zombie ถูกเก็บกวาดทันที ที่ parent ตาย

2.2.2 2.2 Parent ตายแล้ว แต่ Child ยังทำงานอยู่

```
$ gcc -Wall -Wextra -o q2_2 q2_2_parent_first.c
$ ./q2_2 &
[parent] PID = 6857, child PID = 6859 -- exiting immediately
[child ] PID = 6859, PPID = 6857 (original parent, still alive)
[child ] PID = 6859, PPID = 6846 (parent is gone -> re-parented)

$ ps -ef | grep q2_2 | grep -v grep
UID      PID    PPID  C STIME TTY      TIME    CMD
poomipat 6859    6846  0 18:57 pts/2    00:00:00 ./q2_2

$ ps -eo pid,ppid,stat,comm | grep q2_2 | grep -v grep
    PID    PPID STAT  COMMAND
    6859    6846 S+   q2_2      <- normal sleeping state, NO zombie at all
```

Code 8: ผลลัพธ์และสถานะจาก ps ในกรณี 2.2

Table 2: สรุปค่าที่สังเกตได้ในกรณีที่ 2.2

รายการ	ค่าที่สังเกตได้
PID ของ parent	6857
PID ของ child	6859
PPID ของ child (ก่อน parent ตาย)	6857 (parent ตัวจริง)
PPID ของ child (หลัง parent ตาย)	6846 (ถูก reparent)
สถานะของ child ใน ps	S+ (ปกติ ไม่ใช่ zombie)

ตรวจสอบเพิ่มเติมว่า PID 6846 คือ process ใด

```
$ ps -p 6846 -o pid=,ppid=,comm=,args=
6846    6844 Relay(6848) /init

$ # walk the ancestry chain up to PID 1
    PID    PPID COMMAND
    6846    6844 Relay(6848)      <- WSL /init relay : the adopting subreaper
    6844      2 SessionLeader
      2      1 init-systemd(Ub
      1      0 systemd      <- the real PID 1
```

Code 9: สายบรรพบุรุษของ process ที่รับ orphan ไปดูแล

ผลลัพธ์นี้แสดงว่า child **ไม่ได้** ถูกยกให้ PID 1 โดยตรงตามที่มักเข้าใจกัน แต่ถูกยกให้กระบวนการ /init ของ WSL ซึ่งลงทะเบียนตัวเองเป็น *child subreaper* ไปด้วย `prctl(PR_SET_CHILD_SUBREAPER)` เพราะกฎของเคอร์เนลคือยก orphan ให้ **subreaper ที่ใกล้ที่สุด** ไม่ใช่ยกให้ PID 1 เสมอไป บนระบบ Linux ที่ติดตั้งแบบ native หรือใน VM ทั่วไป คำนี้นักออกมาเป็น 1 หรือเป็น PID ของ `systemd --user`

2.3 อภิปรายผล

จากการทดลองควบคุมลำดับการจบการทำงานของกระบวนการด้วย `sleep()` ในข้อ 2.1 และ 2.2 สรุปข้อแตกต่างและประเด็นสำคัญได้ดังนี้

- **ข้อ 2.1 ปรากฏการณ์ Zombie Process:** เมื่อ Child ตายก่อนแต่ Parent ยังทำงานอยู่โดยไม่ได้เรียก `wait()` เคอร์เนลจะคืนหน่วยความจำของ Child ทันที (`SZ = 0`) แต่ยังคงจอง PID และ Process Table Entry ค้างไว้ ทำให้เกิดสถานะ `Z` หรือ `<defunct>` จนกว่า Parent จะจบโปรแกรมหรือเรียก `wait()` เพื่ออ่าน Exit Status
- **ข้อ 2.2 ปรากฏการณ์ Orphan Process และการ Reparent:** เมื่อ Parent ตายก่อนโดยที่ Child ยังคงทำงานอยู่ Child จะกลายเป็นกระบวนการกำพร้า (Orphan) เคอร์เนลจะยก Child ให้กับ Sub-reaper ที่ใกล้ที่สุด (เช่น `systemd` หรือ `/init` ของ WSL) เป็น Parent คนใหม่ทันที สังเกตได้จากค่า PPID ของ Child ที่เปลี่ยนไปในการเรียก `getppid()` ครั้งที่สอง
- **ความแตกต่างสำคัญและปรากฏการณ์ `<defunct>`:** Zombie (`<defunct>`) คือกระบวนการที่สิ้นสุดการทำงานแล้วแต่ยังถูกจองค้างในตารางกระบวนการ หากสะสมมากจะสิ้นเปลือง PID และทำให้สร้าง Process ใหม่ไม่ได้ ต่างจาก Orphan ที่เป็นกระบวนการซึ่งยังคงทำงานตามปกติ และเมื่อ Orphan สิ้นสุดการทำงาน Parent คนใหม่ (Subreaper) จะทำหน้าที่เรียก `wait()` เก็บกวาดให้อัตโนมัติโดยไม่เกิด Zombie ทิ้งไว้

3 จำนวน process สูงสุดที่ fork ได้

[ยังไม่ได้ทำ: ยังไม่ได้เขียนโปรแกรมข้อ 3 – ต้องสร้างไฟล์ q3_forkcount.c]

ข้อควรระวัง ต้องตั้ง `ulimit -u` ก่อนรันทุกครั้ง เช่น `ulimit -u 200` เพื่อไม่ให้ระบบค้าง และควรใช้รูปแบบ recursive ที่ parent `wait()` รอ child เสมอตามที่โจทย์แนะนำ ไม่ใช่การ `fork()` แบบวนลูป ซึ่งจะกลายเป็น fork bomb

3.1 Source code

```
1 /* TODO: recursive form -- parent waits for child, child forks the next
2  * child, until fork() fails. Report the last successful count.
3  */
```

Code 10: q3.c – นับจำนวนครั้งที่ `fork()` สำเร็จ

คำอธิบายการทำงาน [ยังไม่ได้ทำ: อธิบายวิธีนับ กลไกการหยุด และมาตรการป้องกันไม่ให้ระบบค้าง]

3.2 ผลการทดลอง

3.2.1 3.1 การวัดครั้งแรก

```
$ ulimit -u
$ ./q3
```

Code 11: ผลลัพธ์การวัดครั้งแรก

3.2.2 3.2 เมื่อมีโปรแกรมอื่นทำงานค้างอยู่

Code 12: ผลลัพธ์เมื่อรันโปรแกรมอื่นค้างไว้ และหลังปิดโปรแกรมเหล่านั้น

Table 3: เปรียบเทียบจำนวน `fork()` ที่สำเร็จในแต่ละสถานการณ์

สถานการณ์	จำนวน process ที่รันอยู่	<code>fork()</code> สำเร็จ (ครั้ง)
ก่อนรันโปรแกรมเพิ่ม		
ระหว่างรันโปรแกรมเพิ่ม		
หลังปิดโปรแกรมทั้งหมด		

3.3 อภิปรายผล

[ยังไม่ได้ทำ: เขียนอภิปราย] ประเด็นที่ควรกล่าวถึง

- อะไรคือขีดจำกัดที่แท้จริง (`RLIMIT_NPROC` / หน่วยความจำ / ขนาดตารางกระบวนการ)

- `fork()` คืนค่าอะไร และ `errno` เป็นอะไรเมื่อล้มเหลว (EAGAIN หรือ ENOMEM)
- ทำไมตัวเลขจึงเปลี่ยนไปตามจำนวนโปรแกรมที่รันอยู่ และทำไมจึงไม่กลับมาเท่าเดิมเป๊ะ

4 การสื่อสารผ่าน Pipe

[ยังไม่ได้ทำ: ยังไม่ได้เขียนโปรแกรมข้อ 4 – ต้องสร้างไฟล์ q4_pipe.c]

ผลลัพธ์ที่โจทย์กำหนด ต้องได้ output ตามรูปแบบนี้ โดย child เป็นผู้ส่ง และ parent เป็นผู้รับ

```
Child: Child PID: xxxxxx
Parent: Parent PID: yyyyyy
Parent: Hello from child PID: xxxxxx
```

4.1 Source code

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <string.h>
4 #include <sys/wait.h>
5
6 int main(void)
7 {
8     int fd[2];
9     /* TODO: pipe(fd) -> fork()
10      *      child : close(fd[0]) then write()
11      *      parent: close(fd[1]) then read()
12      */
13     return 0;
14 }
```

Code 13: q4.c – child เป็นผู้ส่ง parent เป็นผู้รับ

คำอธิบายการทำงาน [ยังไม่ได้ทำ: อธิบายลำดับ pipe() → fork() → ปิดปลายที่ไม่ใช้ → write() / read() และเหตุผลที่ต้องปิดปลายที่ไม่ใช้ เพราะถ้าไม่ปิดฝั่ง write ให้ครบทุก process แล้ว read() จะไม่มีวันได้รับ EOF และจะ block ค้างตลอดไป]

4.2 ผลการทดลอง

```
$ ./q4
```

Code 14: ผลลัพธ์ของข้อ 4

4.3 อภิปรายผล

[ยังไม่ได้ทำ: เขียนอภิปราย] ประเด็นที่ควรกล่าวถึง เช่น ทำไม pipe จึงเป็นการสื่อสารทางเดียว (uni-directional) ทำไม file descriptor จึงถูกสืบทอดไปยัง child ได้ และ read() มีพฤติกรรม blocking อย่างไร

5 การทดลองกรณีพิเศษของ Pipe

[ยังไม่ได้ทำ: ยังไม่ได้ทำการทดลองข้อ 5 ทั้งหมด – ต้องดัดแปลงโค้ดข้อ 4 เป็นสามเวอร์ชัน]

5.1 5.1 ผู้ส่งพยายามอ่านข้อความของตัวเองกลับ

วิธีทดลอง [ยังไม่ได้ทำ: ให้ child ไม่ปิด fd[0] แล้วลอง read() กลับหลังจาก write() เสร็จ]

```
1 /* TODO */
```

Code 15: ส่วนของโค้ดที่แก้ไขในข้อ 5.1

Code 16: ผลลัพธ์ของข้อ 5.1

สิ่งที่สังเกตได้และการอภิปราย [ยังไม่ได้ทำ: อธิบายว่าสุดท้ายใครได้ข้อมูลไป เพราะ pipe เป็นคิวแบบ FIFO ที่ข้อมูลถูก *อ่านแล้วหมดไป* ไม่ใช่การ broadcast ผู้อ่านคนแรกที่มาถึงจะดึงข้อมูลออกไปอีกฝ่ายจึงไม่เหลืออะไรให้อ่านและอาจ block ต่าง]

5.2 5.2 ผู้รับอ่านก่อนที่ผู้ส่งจะส่ง

วิธีทดลอง [ยังไม่ได้ทำ: ให้ child sleep() สักครู่ก่อน write() เพื่อบังคับให้ parent เรียก read() ก่อนที่จะมีข้อมูลใน pipe]

```
1 /* TODO */
```

Code 17: ส่วนของโค้ดที่แก้ไขในข้อ 5.2

Code 18: ผลลัพธ์ของข้อ 5.2

สิ่งที่สังเกตได้และการอภิปราย [ยังไม่ได้ทำ: อธิบายพฤติกรรม blocking read ว่า read() จะรอจนกว่าจะมีข้อมูล และเงื่อนไขที่ read() จะคืนค่า 0 (EOF) คือเมื่อ write-end ถูกปิดหมดทุก process]

5.3 5.3 ผู้ส่งส่งหลายข้อความก่อนที่ผู้รับจะอ่าน

วิธีทดลอง [ยังไม่ได้ทำ: ให้ child write() หลายครั้งติดกัน แล้ว parent ค่อยเรียก read() ทีหลัง]

```
1 /* TODO */
```

Code 19: ส่วนของโค้ดที่แก้ไขในข้อ 5.3

Code 20: ผลลัพธ์ของข้อ 5.3

สิ่งที่สังเกตได้และการอภิปราย [ยังไม่ได้ทำ: อธิบายว่า pipe เป็น byte stream ที่ไม่รักษาขอบเขตของข้อความ (no message boundary) ข้อความหลายชิ้นจึงถูกอ่านรวมกันมา ใน read() ครั้งเดียวได้ และกล่าวถึงขนาด buffer ของ pipe (ปกติ 64 KB บน Linux) ที่ทำให้ write() block เมื่อ buffer เต็ม]

5.4 สรุปผลการทดลองข้อ 5

Table 4: สรุปพฤติกรรมของ pipe ในแต่ละสถานการณ์

สถานการณ์	ผลที่สังเกตได้	คำอธิบาย
5.1 ผู้ส่งอ่านข้อความตัวเอง		
5.2 ผู้รับอ่านก่อนส่ง		
5.3 ส่งหลายข้อความ ก่อนอ่าน		

สรุปผลการทดลอง

สิ่งที่ได้เรียนรู้จากข้อ 1

1. `fork()` คือการเรียกฟังก์ชันเพียงครั้งเดียวที่คืนค่าสองครั้ง เป็นกลไกพื้นฐานที่ทำให้โค้ดชุดเดียวแตกออกเป็นสองเส้นทางการทำงาน
2. ถ้าไม่มีกลไกชิงโครโนส์ ลำดับการทำงานของ parent กับ child เป็นสิ่งที่ **คาดเดาไม่ได้** และขึ้นกับ scheduler ล้วน ๆ การรันโปรแกรมเดิมซ้ำสามครั้ง ให้ผลไม่ซ้ำกันเลย
3. `wait()` เปลี่ยนโปรแกรมจาก non-deterministic เป็น deterministic และเป็นวิธีป้องกัน zombie process ที่ตรงไปตรงมาที่สุด
4. ได้เห็น **orphan process** และการ reparent โดยบังเอิญจากการรันข้อ 1.1 ซึ่งเป็นปรากฏการณ์เดียวกับข้อ 2.2 ต้องการให้ทดลอง
5. `fork()` คัดลอก **ทุกอย่าง** ใน address space รวมถึง stdio buffer ที่ยังไม่ได้ flush ทำให้เกิดข้อความซ้ำเมื่อ redirect output ลงไฟล์ เป็นกับดักที่พบได้บ่อยและแก้ได้ด้วย `fflush(stdout)` ก่อน `fork()`

ปัญหาที่พบและวิธีแก้

- ข้อความ **"HeLlO WorLd!"** ปรากฏสองครั้ง เมื่อ redirect output ลงไฟล์ ตอนแรกเข้าใจผิดว่าโปรแกรมเขียนผิด ภายหลังพบว่าเกิดจาก stdio buffering แก้โดยเพิ่ม `fflush(stdout)`; ก่อนเรียก `fork()`
- ผลการรันไม่ซ้ำเดิม ทำให้บันทึกผลได้ยาก จึงต้องรันหลายครั้งและรายงานทุกครั้ง เพื่อแสดงให้เห็นความไม่แน่นอน แทนที่จะรายงานเพียงครั้งเดียว

งานที่ยังเหลือ **[ยังไม่ได้ทำ: ข้อ 2, 3, 4 และ 5 – ต้องเขียนโปรแกรม รันเก็บผล และเขียนอภิปรายให้ครบก่อนส่ง]**

เอกสารอ้างอิง

1. Linux manual pages: `fork(2)`, `wait(2)`, `pipe(2)`, `ps(1)`, `setvbuf(3)`.
2. เอกสารประกอบการสอน CPE 333 Operating Systems, บทที่ว่าด้วย Process Management.